

Practical : 03

Aim: Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behaviour by creating orphan and zombie processes.

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- Kill processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

```
Hp@89L1D0M0F305T MINGW64 ~/Desktop/OSCD25D008 (main)
$ taskkill /IM Notepad.exe
ERROR: Invalid argument/option - 'C:/Program Files/Git/IM'.
Type "TASKKILL /?" for usage.
```

```
Hp@89L1D0M0F305T MINGW64 ~/Desktop (master)
$ notepad.exe &
[1] 418

Hp@89L1D0M0F305T MINGW64 ~/Desktop (master)
$ PS
  PID   PPID   PGID   WINPID  TTY      UID     STIME  COMMAND
  400    349    400    15352   pty0     197611  22:49:54 /c/WINDOWS/system32/cmd
  349    348    349    15164   pty0     197611  22:45:50 /usr/bin/bash
  348     1    348    16600   ?        197611  22:45:50 /usr/bin/mintty
  427    402    427    16016   cons0    197611  22:50:50 /usr/bin/PS
  418    402    418    13080   cons0    197611  22:50:44 /c/WINDOWS/system32/notepad
  402     1    402    16672   cons0    197611  22:50:30 /usr/bin/bash

Hp@89L1D0M0F305T MINGW64 ~/Desktop (master)
$ ^C

Hp@89L1D0M0F305T MINGW64 ~/Desktop (master)
$ kill 400

Hp@89L1D0M0F305T MINGW64 ~/Desktop (master)
$ Terminated
```

2. Write a program for process creation using C

▪ Orphan Process

```
GNU nano 8.7 orphan.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid > 0) {
        // Parent process
        printf("Parent process exiting...\n");
        printf("Parent PID: %d\n", getpid());
        sleep(2);
    }
    else if (pid == 0) {
        // Child process
        sleep(5); // parent ke exit hone ka wait
        printf("Child process running...\n");
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID (init/systemd): %d\n", getppid());
    }
    else {
        printf("Fork failed!\n");
    }

    return 0;
}
```

OUTPUT:

```
Nandini Kasare@LAPTOP-NIHKGM40 MSYS ~
$ nano Orphan.c

Nandini Kasare@LAPTOP-NIHKGM40 MSYS ~
$ gcc Orphan.c -o Orphan

Nandini Kasare@LAPTOP-NIHKGM40 MSYS ~
$ ./Orphan
Parent process exiting...
Parent PID: 1183

Nandini Kasare@LAPTOP-NIHKGM40 MSYS ~
$ child process running...
Child PID: 1184
New Parent PID (init/systemd): 1
```

▪ Zombie Process

```
GNU nano 8.7 Zomibe.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        printf("Parent exiting\n");
    }
    return 0;
}
```

Write to File: Zomibe.c

⌘ Help	M-D DOS Format	M-A Append	M-B Backup File	⌘T Browse
⌘C Cancel	M-M Mac Format	M-P Prepend	⌘Q Discard buffer	

OUTPUT:

```
M /c/Users/Hp/Desktop
Hp@89L1D0M0F3O5T MSYS /c/Users/Hp/Desktop
$ nano zombie.c

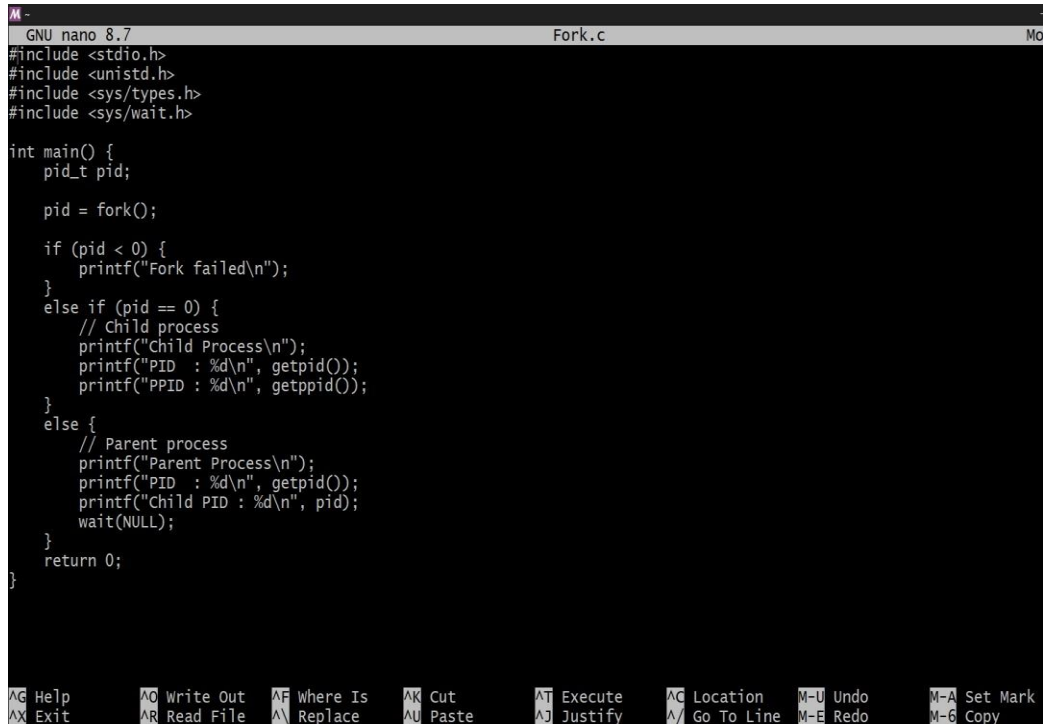
Hp@89L1D0M0F3O5T MSYS /c/Users/Hp/Desktop
$ gcc zombie.c -o zombie

Hp@89L1D0M0F3O5T MSYS /c/Users/Hp/Desktop
$ ./zombie
Child exiting...
Parent still running...

Hp@89L1D0M0F3O5T MSYS /c/Users/Hp/Desktop
$ |
```

3. Create the process using fork () system call.

- ✦ Child Process creation
- ✦ Parent process creation
- ✦ PPID and PID



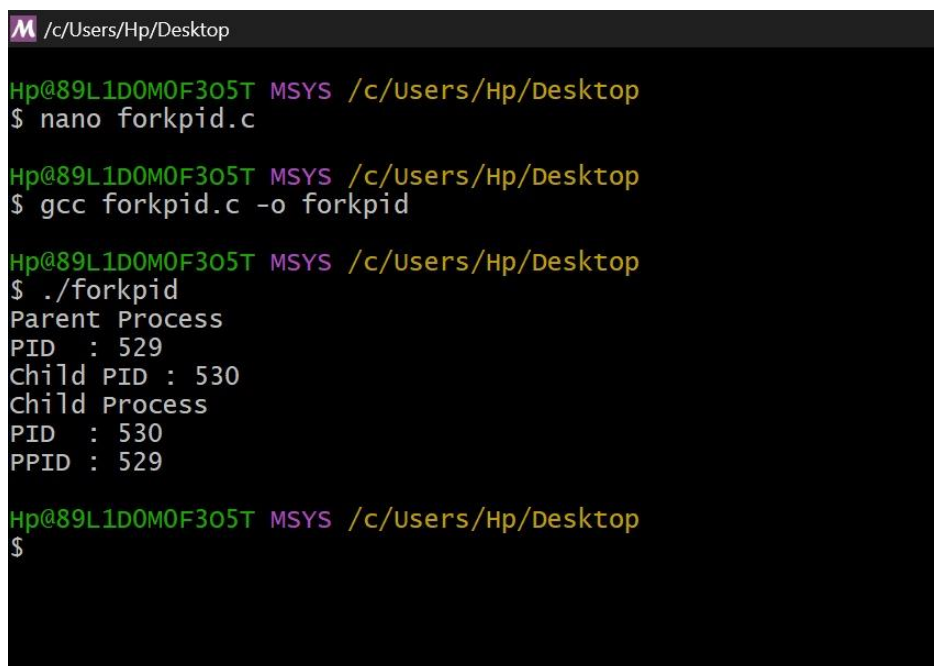
```
GNU nano 8.7 Fork.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        wait(NULL);
    }
    return 0;
}
```

OUTPUT:



```
/c/Users/Hp/Desktop
Hp@89L1D0M0F305T MSYS /c/Users/Hp/Desktop
$ nano forkpid.c

Hp@89L1D0M0F305T MSYS /c/Users/Hp/Desktop
$ gcc forkpid.c -o forkpid

Hp@89L1D0M0F305T MSYS /c/Users/Hp/Desktop
$ ./forkpid
Parent Process
PID : 529
Child PID : 530
Child Process
PID : 530
PPID : 529

Hp@89L1D0M0F305T MSYS /c/Users/Hp/Desktop
$
```